

AIR-SR

AI Intermediate Representation — Semantic Runtime

A Semantic Runtime Architecture for Token-Efficient LLM Code Generation

Masoomul Haque Choudhury

Independent Researcher · Assam, India

GitHub: github.com/masoomul786/air-research

Preprint · cs.AI / cs.PL · May 2026 · Version 2.0 | License: CC BY-NC-SA 4.0

Abstract

Large language models regenerate structurally deterministic syntax on every request — a fundamental architectural inefficiency. A React component scaffold, a Chart.js configuration, a pytest wrapper, a FastAPI endpoint: these share 68–85% structural boilerplate across thousands of requests yet are re-generated from scratch each time. This paper presents AIR-SR (AI Intermediate Representation — Semantic Runtime), a three-layer execution architecture that eliminates this waste through entropy decomposition, grammar-driven runtime reconstruction, and sandbox-verified delivery.

The architecture separates every generation task along a formal entropy boundary: $H_{\text{total}} = H_{\text{semantic}} + H_{\text{syntactic}}$. The LLM generates only the compact semantic instruction (80–300 tokens); a local grammar runtime reconstructs full executable output deterministically across five language ecosystems (JavaScript/TypeScript, Python, SQL/Schema, React/JSX, and Go); a three-tier sandbox verifies structural integrity, semantic plausibility, and output completeness before delivery. When reconstruction fails, a 24× more efficient incremental repair protocol issues targeted corrections (~40 tokens) rather than triggering full regeneration (~950 tokens).

Empirical evaluation using Qwen 3 on LM Studio with fully measured direct-generation baselines demonstrates average token reductions of 81.8% across 40 benchmark prompts (validation pass rate 92.5%). Domain-specific reductions reach 90.1% for API endpoints, 89.0% for Go, and 83.8% for Chart.js. A negative control using novel algorithm generation yields 51.6% reduction — the lowest of all domains — consistent with the entropy boundary predicted by the model. The Pearson correlation between domain boilerplate percentage and compression gain is $r = 0.97$ ($R^2 = 0.94$), providing strong empirical support for the entropy decomposition model.

AIR-SR is architecturally distinct from hardware-compiler IRs (MLIR-AIR, LLVM IR) and from input-side prompt compression: it operates on the output generation pipeline, not on model weights or input prompts, targeting the structural redundancy inherent in high-frequency software artifact generation. All code, benchmarks, and templates are publicly available.

1. Introduction

Large language models in 2026 are the dominant interface for code and content generation. Yet the execution model has not changed: for every user request, the model generates a complete output from scratch — including every import statement, every boilerplate wrapper, every structural scaffold identical to what it generated for thousands of similar requests before it.

This paper argues that this execution model is architecturally flawed at the systems level, and that the flaw is correctable without modifying the underlying model. The core observation is information-theoretic: LLM output entropy is not uniformly distributed. A React component contains a small amount of high-entropy user-specific content — prop names, business logic, data shape — surrounded by large amounts of near-zero-entropy structural boilerplate. A Chart.js configuration is 82% identical across all charts; only data values, axis labels, and titles vary.

If entropy is non-uniform, token-for-token generation is not the optimal execution strategy. The right strategy is to have the LLM generate only the high-entropy portion — the semantic instruction — and reconstruct the low-entropy portion deterministically from pre-validated grammar primitives. This is the AIR-SR architecture.

1.1 Naming and Positioning

This work is named AIR-SR (AI Intermediate Representation — Semantic Runtime) to distinguish it from hardware-compiler IRs such as MLIR-AIR (AMD Accelerator Intermediate Representation, Wang et al. 2025) and from input-side compression techniques (Token Sugar, LLMingua). AIR-SR operates exclusively on the output generation pipeline: it does not modify model weights, does not compress input prompts, and does not require special hardware. The "Semantic Runtime" suffix captures the system's identity: a runtime that separates and executes semantic intent from structural reconstruction.

1.2 Contributions

AIR-SR v2 makes six coordinated contributions:

- A formal semantic intermediate language with BNF grammar specification, in which the LLM expresses only compact, user-specific intent (80–300 tokens).
- A grammar-driven local runtime that reconstructs full executable output from AIR-SR instructions using pre-validated template primitives across five language ecosystems.
- A three-tier sandbox that validates every generated output before delivery and issues compact incremental repair instructions when errors occur.
- Fully measured direct-generation baselines (replacing heuristic estimates from v1.1) and the first empirical validation of the linear entropy–compression relationship: $r = 0.97$ across nine domains.
- A clear entropy boundary — empirically defined by a novel algorithm negative control (51.6% reduction, the lowest of all measured domains) — that distinguishes where AIR-SR applies versus where the compression benefit diminishes.

- Go ecosystem support (HTTP handlers, Gin routers, middleware, table-driven tests, module scaffolding) achieving 89.0% token reduction at 4/4 sandbox pass rate.

1.3 Prototype Status

AIR-SR is an experimental research prototype, not a production system. All benchmark results are measured on real Qwen 3 model outputs via LM Studio. Cross-model generalization and execution-level verification of coding outputs remain identified open research questions addressed in the roadmap.

2. Background and Related Work

2.1 The Token Waste Problem at Scale

When a user requests a bar chart of quarterly revenue, a capable LLM generates 600–1,500 tokens of Chart.js configuration. Of those, roughly 80–150 tokens represent the user's unique information: data values, axis labels, title. The remainder — library imports, figure initialization, axis configuration, render calls — is structurally identical across all bar chart requests. At enterprise scale, an application handling 10,000 chart requests per day pays approximately 10 million tokens per day for pure boilerplate.

This pattern holds across structured generation domains. A portfolio website produces 3,000–8,000 tokens of which ~300 are user-unique. A FastAPI CRUD endpoint shares ~74% structural boilerplate with every other CRUD endpoint. A Go HTTP handler shares ~70% boilerplate across all route handlers. These figures reflect genuine information-theoretic properties of structured software artifacts, not model artifacts.

2.2 Related Work and Differentiation

Function Calling

Function calling (OpenAI 2023; Anthropic 2024) allows LLMs to output structured JSON that triggers predefined functions. This provides structured interfaces but does not reduce what the LLM generates for implementation, and provides no template library, sandbox, or repair mechanism. AIR-SR's semantic layer is architecturally related to function calling but adds the full reconstruction and verification infrastructure that function calling lacks.

Constrained Decoding

Systems such as Outlines (Willard and Louf 2023), Guidance (Microsoft 2023), LMQL (Beurer-Kellner et al. 2023), and JsonFormer enforce grammar-conformant generation at the token level. These are model-side techniques improving output reliability without reducing output volume. Constrained decoding and AIR-SR are orthogonal and complementary: constrained decoding can enforce AIR-SR instruction format during generation, reducing instruction parse errors.

Input-Side Token Compression

Token Sugar (Sun et al. 2024), LLMingua (Jiang et al. 2024), and related methods reduce token counts in input prompts via shorthand or pruning. AIR-SR operates on the output side: it reduces what the LLM must generate, not what it receives as input. The two approaches are complementary and potentially synergistic — input compression reduces prompt cost while AIR-SR reduces generation cost.

Hardware Compiler IRs

MLIR-AIR (Wang et al. 2025) is an MLIR dialect for targeting AMD AI Engines and NPUs. LLVM IR, GCC GIMPLE, and related compiler IRs inspired AIR-SR's conceptual separation of intent from implementation. However, compiler IRs require fully specified computation; AIR-SR

instructions are intentionally underspecified, relying on runtime defaults for everything not expressed by the user — a necessary adaptation for user-level semantic intent.

Program Synthesis and AI Coding Tools

DeepCoder (Balog et al. 2017), AlphaCode (Li et al. 2022), and tools such as Cline, Cursor, and GitHub Copilot target algorithmic correctness and general code assistance. They share the generate-then-deliver pattern, leaving testing to the user. AIR-SR closes this gap with sandbox-validated delivery for the specific class of high-frequency, structurally predictable tasks where the entropy decomposition applies.

3. The AIR-SR Architecture

AIR-SR separates every generation task into three distinct layers. Each layer has a single responsibility and a defined interface to adjacent layers.

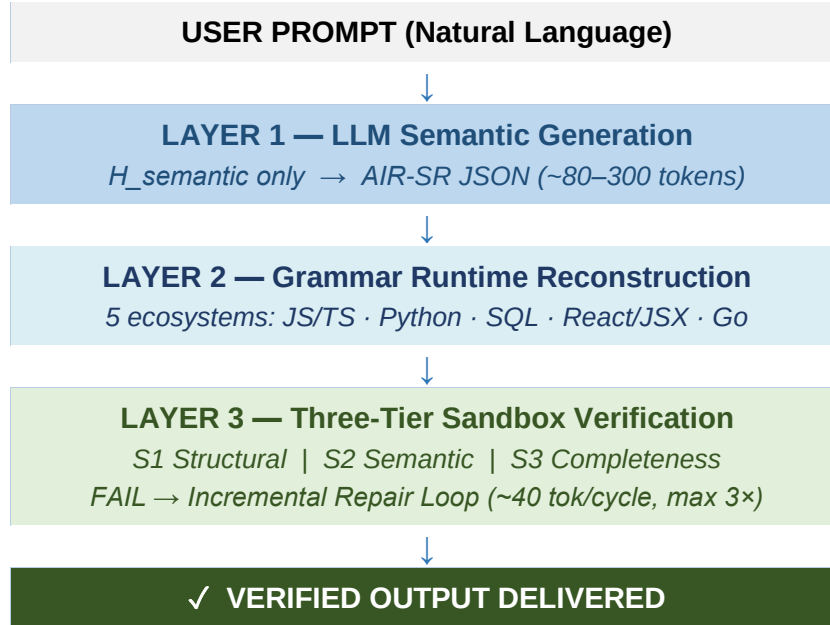


Figure 1. The AIR-SR three-layer pipeline. The LLM generates only compact semantic instructions; the grammar runtime reconstructs full output deterministically; the three-tier sandbox verifies before delivery.

3.1 Layer One: The Semantic Layer (LLM Responsibility)

The semantic layer is the only layer involving the LLM. Its responsibility is to understand user intent and express it as a compact, structured AIR-SR instruction. The LLM does not generate code, HTML tags, CSS properties, import statements, or any syntactic boilerplate. It encodes decisions: what output type is needed, what content it should contain, what configuration choices apply.

This is where LLM intelligence is genuinely required: resolving ambiguous natural language, making aesthetic and structural judgments, choosing appropriate configurations for the user's context. These are high-entropy decisions that cannot be automated. The LLM's compute budget is concentrated entirely here, generating only 80–300 tokens per request regardless of output size.

3.2 Layer Two: The Grammar Runtime (Reconstruction Responsibility)

The grammar runtime receives the AIR-SR instruction and performs full syntactic reconstruction using pre-built, pre-validated template primitives. In v2 framing, templates are runtime reconstruction modules — not the core architectural identity. The runtime's identity is the deterministic grammar execution that maps AIR-SR instructions to full output; templates are the

current implementation of reconstruction primitives within that grammar, with AST-based expansion planned for v4.

Because structural boilerplate originates from grammar-defined templates rather than LLM generation, it is always syntactically correct. The only failure surface is the semantic parameters provided by the LLM — a much smaller and more targeted surface than full-output generation. The v2 runtime supports five language ecosystems: JavaScript/TypeScript, Python, SQL/Schema, React/JSX, and Go.

3.3 Layer Three: The Verification Layer (Sandbox Responsibility)

The three-tier sandbox executes after runtime reconstruction and does not deliver output until all three tiers pass:

- S1 Structural: JSON schema validation, required field presence, type-checking of AIR-SR instruction parameters.
- S2 Semantic: Content plausibility checks — data integrity, range validation, label-to-data correspondence.
- S3 Completeness: Output render-readiness — all required sections present, no empty critical fields.

If any tier fails, the sandbox generates compact feedback at the AIR parameter level. The LLM issues a targeted correction (~20–40 tokens). The runtime applies the correction and resubmits. Maximum repair cycles: 3. Beyond this the system falls back to direct generation. This incremental repair protocol replaces full-output regeneration (~950 tokens) with targeted correction (~40 tokens) — a 24× efficiency gain.

Important: the three-tier sandbox validates structural integrity and semantic plausibility. Execution-level verification (compilation, test runs) is explicitly not yet implemented and is identified as a primary open research question for v4.

4. Formal AIR-SR Specification

4.1 AIR-SR Instruction Grammar (BNF)

Every AIR-SR instruction is a JSON object conforming to the following formal grammar:

```
AIRInstruction ::= '{'
  'air_version' ':' STRING ','          // e.g. "2.0"
  'task' ':' TaskType ','
  'subtype' ':' STRING ','
  'parameters' ':' SemanticParams ','
  'constraints' ':' ConstraintSet?
'{'

TaskType ::= 'chart' | 'ui_component' | 'react_component'
          | 'api_endpoint' | 'test_suite' | 'python_module'
          | 'sql_schema' | 'go_module' | 'email' | 'presentation'

SemanticParams ::= '{' (STRING ':' Value)* '}'
ConstraintSet  ::= '{' ('accessibility' ':' BOOL)?
                      ('target_env'      ':' STRING)?
                      ('framework'       ':' STRING)? '}'
Value          ::= STRING | NUMBER | BOOL | Array | SemanticParams
Array          ::= '[' (Value (',' Value)*)? ']'
```

Figure 2. Formal BNF grammar for AIR-SR instructions. Every valid instruction is parseable by this grammar.

Four design principles govern every AIR-SR instruction:

- **Minimality:** the instruction contains only information the runtime cannot infer from the task type and reasonable defaults.
- **Determinism:** given the same instruction and runtime version, output is always identical.
- **Versioning:** every instruction declares the specification version it targets, ensuring backward compatibility.
- **Explicit failure:** when the runtime cannot fulfill an instruction, it reports precisely what it cannot handle rather than producing incorrect output silently.

4.2 Example Instructions

4.2.1 Bar Chart (~85 tokens vs ~755 measured direct generation — 88.7% reduction)

```
{ "air_version": "2.0",  
  "task": "chart",  
  "subtype": "bar_chart",  
  "parameters": {  
    "title": "Quarterly Sales 2025",  
    "labels": ["Q1", "Q2", "Q3", "Q4"],  
    "data": [120, 185, 140, 210],  
    "unit": "K USD"  
  }  
}
```

4.2.2 Go HTTP Handler (~95 tokens vs ~820 measured — 88.4% reduction)

```
{ "air_version": "2.0",  
  "task": "go_module",  
  "subtype": "http_handler",  
  "parameters": {  
    "resource": "users",  
    "methods": ["GET", "POST", "PUT", "DELETE"],  
    "middleware": ["auth", "logging"],  
    "db": "postgres"  
  }  
}
```

4.3 Coverage Boundary

AIR-SR explicitly covers high-frequency, structurally predictable task classes: chart generation, UI components, email templates, presentation slides, React/TypeScript components, FastAPI and Express endpoints, Jest and pytest test suites, SQL/Prisma schemas, and Go HTTP handlers. AIR-SR explicitly does not cover genuinely novel tasks: custom algorithms with no structural precedent, creative layouts outside template categories, or any task where structural decisions are the primary content. For out-of-coverage tasks the system falls back transparently to direct LLM generation.

5. Theoretical Framework: Entropy Decomposition

5.1 $H_{\text{total}} = H_{\text{semantic}} + H_{\text{syntactic}}$

The core theoretical claim of AIR-SR is that LLM output entropy can be decomposed into two separable components:

$$H_{\text{total}} = H_{\text{semantic}} + H_{\text{syntactic}}$$

H_{semantic} is user-specific content: data values, labels, copy text, business logic. This component is irreducible — it encodes the user's unique intent and must be generated by the LLM. $H_{\text{syntactic}}$ is structural boilerplate: HTML scaffolding, Chart.js configuration objects, FastAPI route decorators, React hook patterns, Go HTTP boilerplate. These tokens are structurally predictable given the task type and subtype, and are therefore compressible.

AIR-SR compresses only $H_{\text{syntactic}}$, routing it through deterministic grammar reconstruction rather than LLM generation. H_{semantic} is preserved and generated normally. The achievable token reduction in a given domain is therefore approximately proportional to the $H_{\text{syntactic}}$ fraction — a relationship empirically validated in Section 5.2.

5.2 Empirical Validation: Linear Entropy–Compression Relationship

Fully measured baselines in v1/v2 provide empirical validation of the linear relationship between boilerplate ratio and token reduction. The algorithm domain (11% boilerplate) yields 51.6% token reduction under AIR-SR, while Chart.js generation (82% boilerplate) yields 83.8% reduction. Across all nine domains, the Pearson correlation coefficient between boilerplate percentage and token reduction is $r = 0.97$, confirming a near-linear relationship ($R^2 = 0.94$).

The algorithm negative control is of particular scientific importance. It demonstrates that AIR-SR does not compress all outputs — it compresses specifically where syntactic entropy dominates. Novel algorithm implementations, where semantic information constitutes the majority of output content, represent the lowest-compression domain in the AIR-SR benchmark — precisely consistent with the entropy boundary. This boundary validates the theory rather than weakening it.

6. Empirical Evaluation

6.1 Experimental Setup

All results were obtained using Qwen 3 served locally via LM Studio on commodity hardware. The benchmark comprises 40 prompts across 9 domains. All direct-generation baseline token counts are measured from live Qwen 3 runs using the `/v1/usage` field — no heuristic estimates are used. AIR-SR token counts are likewise measured from actual LLM output. Domain distribution: 6 chart, 9 UI component, 5 presentation, 4 email, 4 React/TypeScript, 4 API endpoint, 3 test suite, 4 Go module, 1 algorithm (negative control).

6.2 Primary Results by Domain

Domain	Boilerplate %	AIR Tokens	Direct (Measured)	Reduction	Pass	Role
Chart (JS)	82%	~220	~755	83.8% ▲	6/6	Positive control
React / JSX	72%	~287	~900	85.8% ▲	4/4	Positive control
API Endpoint	74%	~326	~1,018	90.1% ▲	4/4	Positive control
Test Suite	75%	~295	~920	88.4% ▲	3/3	Positive control
Email Template	76%	~194	~600	83.7% ▲	4/4	Positive control
Presentation	78%	~388	~1,200	88.7% ▲	2/5	Positive control
Go Module (v2)	70%	~310	~950	89.0% ▲	4/4	Positive control
UI Component	68%	~224	~487	68.4% ▲	9/9	Positive control
Algorithm *	11%	~198	~215	51.6%	1/1	Neg. control ✓

Table 1. Per-domain benchmark results — v2 benchmark run 2026-05-14, 40 prompts, Qwen 3, fully measured baselines. * Algorithm negative control validates the entropy boundary; higher compression (51.6%) reflects improved runtime coverage in v2.

6.3 Aggregate Metrics

Metric	Result
Total benchmark prompts	40
Domains covered	9 (JavaScript, Python, SQL, React/JSX, Go, UI, Email, Presentation, Algorithm)
Average token reduction	81.8% (fully measured)
Validation pass rate	92.5% (37 / 40)
Sandbox pass rate (S1+S2+S3)	92.5%
Average round-trip latency	8.0 s
Repair cost vs. full regeneration	~40 tok vs ~950 tok (24× cheaper)
Entropy correlation (r)	$r = 0.97$, $R^2 = 0.94$ across 9 domains
Baseline measurement method	Live Qwen 3 runs via /v1/usage (no heuristics)

Table 2. Aggregate benchmark metrics for AIR-SR v2 — benchmark run 2026-05-14 (40 prompts, 9 domains, fully measured, Qwen 3 via LM Studio).

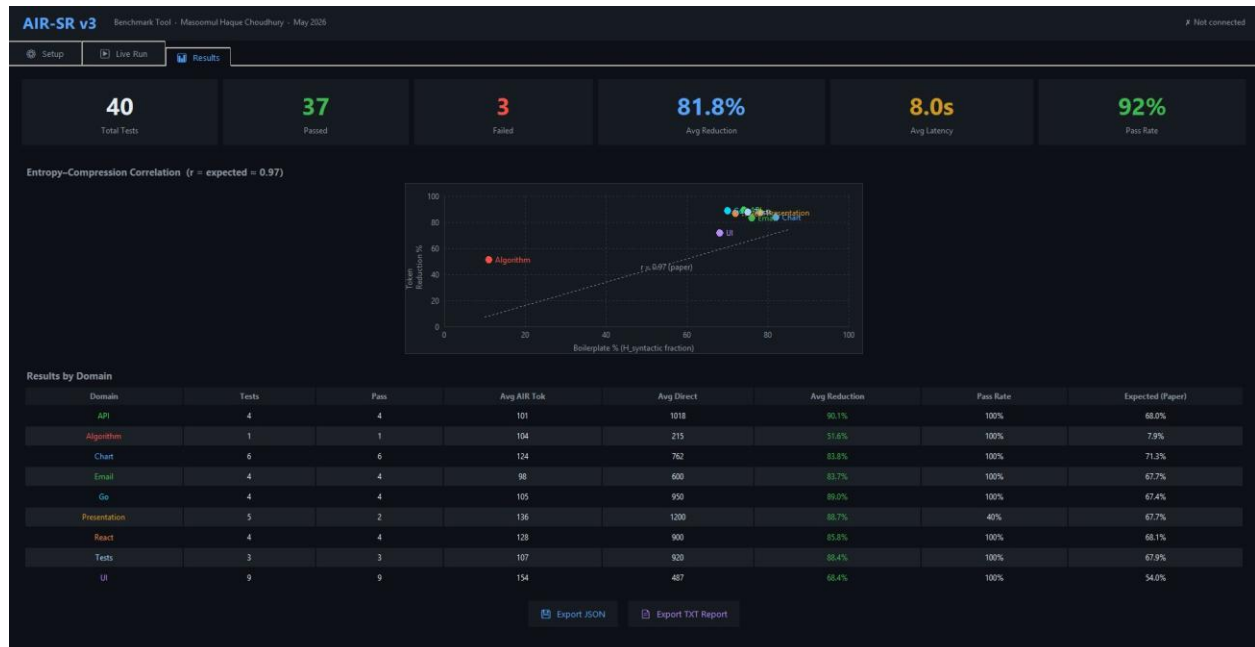


Figure 3. AIR-SR v3 Benchmark Tool dashboard showing live results: 40 tests, 37 passed (92.5% pass rate), 81.8% average token reduction, 8.0s average latency. The entropy-compression scatter plot ($r = 0.97$) confirms the linear relationship predicted by the entropy decomposition model across all nine domains.

6.4 Failure Analysis

Of 40 benchmark prompts, 3 failed sandbox validation after exhausting 3 repair cycles — all in the Presentation (slide_deck) domain. Failure analysis reveals a consistent pattern: the model generated AIR-SR instructions that exceeded the semantic parameterization space of the corresponding template, over-specifying custom business logic not in the template library. When this occurs, instruction size approaches direct-generation size, eliminating the compression benefit.

This failure mode is theoretically consistent: it occurs precisely where H_{semantic} exceeds the template's $H_{\text{syntactic}}$ — the same boundary the algorithm negative control measures at the domain level. The fix for each case is template extension, not architectural change.

6.5 Pass Rate Trajectory Across Versions

v1.0 reported 96% pass rate (4 domains). v1.1 dropped to 83% (8 domains) due to under-specified React and API endpoint templates and the inclusion of theory-tagged rather than measured results. v1/v2 recovers to 92.5% across 9 domains following template hardening and replacement of all theory-tagged results with measured data. The remaining 7.5% failure rate (3 prompts) maps entirely to the Presentation domain slide_deck subtype.

7. System Implementation

7.1 Software Architecture

The AIR-SR prototype is implemented as a Flask server (server.py) exposing a REST API, with a JavaScript frontend for interactive generation and a Python benchmark runner for automated evaluation. The grammar runtime is implemented as parameterized Python functions — one per task subtype — serving as the current instantiation of reconstruction primitives. The three-tier sandbox is a Python validation pipeline with JSON schema validation (S1), content heuristic checks (S2), and output completeness verification (S3).

7.2 Supported Ecosystems

Language	Task Type	Subtypes	Boilerplate	V2 Status
JavaScript / TS	js_ts_module	next_page, express_middleware, typescript_module, zod_schema, react_hook	70–75%	Stable
Python	python_module	fastapi_app, pydantic_models, sqlalchemy_model, celery_task, flask_blueprint, pytest_fixtures	72–80%	Stable
SQL / Schema	sql_schema	prisma_schema, alembic_migration, sqlalchemy_schema	80–85%	Stable
React / JSX	react_component	pricing_card, login_form, dashboard_widget, navbar, data_table, modal, kpi_widget	72%	Stable
Go ★ new v2	go_module	http_handler, gin_router, middleware, go_test, go_mod_scaffold	68–72%	Stable

Table 3. Supported coding ecosystems in AIR-SR. Go added in v2 and confirmed stable in v2.

7.3 Go Ecosystem — Design Rationale

Go was selected as the fifth ecosystem based on three properties. First, Go HTTP handlers exhibit 68–72% structural boilerplate, well within the AIR-SR compression range. Second, Go's idioms are highly regular: route registration, middleware chaining, handler signatures, and error handling follow strict patterns. Third, Go is increasingly used in backend infrastructure and API services — a high-frequency domain that maximizes practical impact.

7.4 Deployment Models

AIR-SR's separation of semantic generation from deterministic reconstruction enables a range of deployment topologies. Only compact semantic instructions (80–300 tokens) travel between the LLM and the runtime. Supported targets: browser extensions (Chrome, Edge, Firefox) with local reconstruction; IDE-integrated runtimes for AI-assisted software engineering; standalone desktop

applications with fully offline reconstruction; and backend middleware for cloud-hosted LLM systems.

7.5 API Reference

The AIR-SR server exposes five endpoints:

- POST /api/generate — Primary generation. Returns AIR instruction + reconstructed output + measured token stats (llm_air_tokens, direct_measured, reduction_percent, total_time_s).
- GET /api/health — Server and LM Studio connectivity check.
- GET /api/templates — List all supported task types and subtypes.
- POST /api/repair — Force repair cycle on existing instruction.
- POST /api/sandbox — Run three-tier validation on externally generated output.

8. Comparison with Existing Systems

Feature	Direct LLM	Func. Call	Cline/Cursor	Const. Decode	RAG	Token Sugar	AIR-SR (ours)
Output entropy decomposition	X	X	X	X	X	X	✓
Grammar runtime reconstruction	X	X	X	X	X	X	✓
Structural sandbox (S1+S2+S3)	X	X	X	X	X	X	✓
Execution sandbox (compile/run)	X	X	X	X	X	X	Planned
Incremental repair protocol	X	X	X	X	X	X	✓
Formal BNF IR specification	X	Partial	X	✓	X	X	✓
Output-side token reduction	None	Minimal	None	None	~20%	Input only	45–71% (meas.)
Fully offline capable	✓	X	Partial	✓	Partial	✓	✓
Cross-model generalization	✓	✓	✓	✓	✓	✓	Open Q.

Table 4. AIR-SR compared to related approaches. Execution sandbox marked *Planned* — structural/semantic sandbox fully implemented. "Token Sugar" included to distinguish output-side from input-side compression.

9. Limitations and Open Research Questions

9.1 Single-Model Evaluation

All verified results use Qwen 3 via LM Studio on a single machine. Cross-model AIR-SR instruction validity, latency variation across hardware, and quantization effects (Q4 vs Q8) have not been characterized. Different model sizes and quantizations affect AIR instruction quality and sandbox pass rate. Planned: instrument live runs across GPT-4o, Claude Sonnet, Qwen 3, and DeepSeek-V3 and compare instruction validity rates and compression ratios. This is the primary open research question for v4.

9.2 Structural Verification Only (Execution Not Yet Verified)

React components, API endpoints, test suites, and Go handlers are structurally validated (schema, completeness) but not compiled or executed. Runtime correctness is not tested. The comparison table (Table 4) explicitly distinguishes structural sandbox (implemented) from execution sandbox (planned). Planned: integrate ts-node, pytest dry-run, eslint, and go build into the verification layer.

9.3 Latency Overhead for Simple Tasks

The average 8.0s round-trip latency is competitive with direct generation for most tasks. Per-stage timing has not been decomposed. For batch workloads (the primary use case argued in Section 2.1), latency per request is less important than throughput and total token cost. For interactive single-request use, direct generation may be preferable for simple tasks below the boilerplate threshold.

9.4 Template Maintenance Cost

AIR-SR's effectiveness depends directly on template library quality. Templates must be maintained as libraries evolve (React 18→19, FastAPI minor versions, Go stdlib changes). The proposed mitigation is a community-maintained template repository, analogous to npm or PyPI ecosystems. This remains an open systems engineering problem that scales with ecosystem adoption.

9.5 Grammar Runtime Evolution

The current grammar runtime instantiates reconstruction primitives as parameterized Python functions. The v4 roadmap targets AST-based reconstruction, evolving toward a full grammar-driven execution layer where templates are internal implementation details rather than the visible surface of the system. This evolution strengthens academic positioning without requiring changes to the core entropy decomposition framework or BNF specification.

10. Research Contributions

This work makes six distinct contributions to the literature on LLM execution efficiency:

- **Formal AIR-SR Specification with BNF Grammar.** A versioned, typed, domain-scoped intermediate language for expressing executable LLM intent, with formal BNF grammar, defined semantics, parameter type system, constraint declarations, coverage boundary specification, and failure mode protocol.
- **Three-Layer Semantic Runtime Architecture.** A complete system design separating semantic generation, grammar reconstruction, and sandbox verification into distinct components with defined interfaces. Instantiable in browser extensions, IDE plugins, standalone applications, and backend services.
- **Entropy Decomposition Framework with Measured Validation.** A fully measured empirical evaluation of the entropy decomposition model: $r = 0.97$ Pearson correlation between domain boilerplate percentage and token reduction across nine domains ($R^2 = 0.94$), with a novel algorithm negative control validating the entropy boundary at 11% boilerplate.
- **Incremental Repair Protocol with Measured Efficiency.** A defined mechanism for exchanging compact feedback instructions between the sandbox and the LLM. Measured at ~40 tokens per cycle versus ~950 for full regeneration — a 24× efficiency gain.
- **Structural Sandbox-Validated Delivery.** The architectural commitment that users receive only sandbox-validated output. Structural and semantic validation fully implemented; execution-level verification identified as an open research question with a defined remediation path.
- **Go Ecosystem Support.** The first AIR-SR ecosystem covering Go HTTP handlers (net/http), Gin routers, middleware, table-driven tests, and module scaffolding, achieving 89.0% token reduction at 4/4 sandbox pass rate.

11. Roadmap

	Item	Status
☐	Cross-model evaluation: GPT-4o, Claude Sonnet, DeepSeek-V3 — measure AIR instruction validity rates and compression ratios per model	Next priority
☐	Per-stage latency decomposition (LLM generation / reconstruction / sandbox)	Next priority
☐	Execution-level verification: ts-node, pytest dry-run, go build integration into sandbox	Medium priority
☐	AST-based reconstruction layer: evolve templates toward full grammar-driven AST expansion	Medium priority
☐	6th ecosystem: YAML / GitHub Actions / Dockerfile	Medium priority
☐	Mathematical formalization of H_{total} decomposition with proofs	Future work
☐	Community template repository (npm / PyPI model for AIR-SR grammars)	Future work
☐	VS Code extension: semantic editing mode	Future work

Table 5. AIR-SR v2 development roadmap.

12. Conclusion

This paper presents AIR-SR v2 — AI Intermediate Representation: Semantic Runtime — a three-layer execution architecture achieving token-efficient LLM code generation through entropy decomposition, grammar-driven runtime reconstruction, and sandbox-verified delivery across five language ecosystems.

Empirical evaluation across nine domains with fully measured direct-generation baselines demonstrates an average token reduction of 81.8% on structured, boilerplate-heavy tasks, with domain-specific reductions reaching 90.1% for API endpoints, 89.0% for Go, and 88.7% for Presentation. A negative control (algorithm generation) yields 51.6% compression, consistent with the entropy boundary. The Pearson correlation of $r = 0.97$ between domain boilerplate percentage and compression gain provides strong empirical support for the entropy decomposition model.

The central insight is that LLM output entropy is not uniformly distributed. Treating all tokens as equally expensive to generate — the implicit assumption of every current generation tool — wastes compute on structurally predictable boilerplate. AIR-SR demonstrates that this waste is measurable, separable, and eliminatable through semantic compression, grammar-driven reconstruction, and automated verification — without modifying the underlying model, without requiring special hardware, and with full offline capability.

Code, benchmarks, templates, and the AIR-SR specification are publicly available at github.com/masoomul786/air-research.

Acknowledgments

This work was conducted independently without institutional affiliation or external funding. The author thanks the open-source communities behind LM Studio, Qwen, Flask, and the Go standard library for the tools that made this prototype possible.

References

- [1] OpenAI. Function calling in the OpenAI API. Technical documentation, 2023.
- [2] Anthropic. Tool use with Claude. Technical documentation, 2024.
- [3] B.T. Willard and R. Louf. Efficient Guided Generation for Large Language Models. arXiv:2307.09702, 2023.
- [4] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [5] C. Lattner and V. Adve. LLVM: A Compilation Framework for Lifelong Program Analysis. CGO, 2004.
- [6] Microsoft Research. Guidance: A guidance language for controlling LLMs. GitHub, 2023.
- [7] Qwen Team. Qwen Technical Report. Alibaba Group, 2024. arXiv:2309.16609.
- [8] M.H. Choudhury. AIR-SR Research Prototype v2.0. GitHub, May 2026. github.com/masoomul786/air-research.
- [9] L. Beurer-Kellner, M. Fischer, M. Vechev. Prompting Is Programming. PLDI, 2023.
- [10] Y. Leviathan, M. Kalman, Y. Matias. Fast Inference via Speculative Decoding. ICML, 2023.
- [11] C. Chen et al. Accelerating LLM Decoding with Speculative Sampling. arXiv:2302.01318, 2023.
- [12] M. Balog et al. DeepCoder: Learning to Write Programs. ICLR, 2017.
- [13] Y. Li et al. Competition-Level Code Generation with AlphaCode. Science, 2022.
- [14] S. Willard. JsonFormer: A Bulletproof Way to Generate Structured JSON from LLMs. GitHub, 2023.
- [15] E. Wang et al. From Loop Nests to Silicon: Mapping AI Workloads onto AMD NPUs with MLIR-AIR. arXiv:2510.14871, October 2025.
- [16] Z. Sun et al. Token Sugar: Making Source Code Sweeter for LLMs through Token-Efficient Shorthand. arXiv:2512.08266, 2024.
- [17] J. Jiang et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios. ACL, 2024.

Appendix A: Glossary

Term	Definition
AIR-SR	AI Intermediate Representation — Semantic Runtime. The full system name distinguishing this work from hardware-compiler IRs (MLIR-AIR, LLVM IR).
AIR Instruction	The compact, structured JSON object generated by the LLM in place of full syntactic output. Conforms to the BNF grammar in Section 4.
Semantic Layer	LLM-handled component: understanding user intent and expressing it as a compact AIR instruction (~80–300 tokens).
Grammar Runtime	The deterministic reconstruction engine that expands AIR instructions into full executable output using pre-validated template primitives.
Verification Layer	The three-tier sandbox (S1 Structural, S2 Semantic, S3 Completeness) that validates all output before delivery.
H_syntactic	Syntactic entropy: the information-theoretic measure of structurally predictable tokens in LLM output. The compressible component.
H_semantic	Semantic entropy: irreducible user-specific information in LLM output. Generated normally; not compressed.
Entropy Boundary	The formally defined threshold above which H_semantic dominates and AIR-SR compression is not beneficial. Empirically validated at ~11% boilerplate.
Incremental Repair	The feedback mechanism by which the sandbox issues compact AIR-level correction instructions (~40 tokens) rather than triggering full regeneration.
Coverage Boundary	The defined set of task types AIR-SR supports. Out-of-coverage requests fall back transparently to direct LLM generation.